Circles Invitation Escrow Audit

Gnosis Ltd - Report by Côme du Crest

2025-06-11

Table of contents

- Table of contents
- Circles Invitation Escrow Audit
  - Scope
  - Context
  - Status
- Issues
  - [Low] Unsafe assembly block

Circles Invitation Escrow Audit

This document presents the findings of a smart contract audit conducted by Côme du Crest for Gnosis Ltd.

Scope

The scope includes all contracts within `aboutcircles/circles-invitation-escrow` as of commit 0x9f1e792.

The code was updated to fix one issue and was reviewed as of commit `0x4dc2fa3`.

Context

The goal of the contracts is to escrow CRC tokens for invitations so that inviters and invitees are guaranteed enough inviter's personal CRCs are available for onboarding.

Status

The report has been sent to the core developer.

The code has been fixed and no issue remain.

Issues

[Low] Unsafe assembly block

Summary

The function `_getInviteesOrInviters()` uses unsafe assembly that manipulates the memory layout and breaks solidity's assumption on memory. It still works due to the way it is used and seemingly luck.

Vulnerability Detail

The internal function `_getInviteesOrInviters()` attempts to populate a dynamic array by increasing its size stored at its initial memory location and by increasing the free memory pointer stored at `0x40`:

```
1      function _getInviteesOrInviters(address inviterOrInvitee)
2          internal
3          view
4          returns (address[] memory inviteesOrInviters)
5      {
6          address previous = invitationLinkedList[inviterOrInvitee][SENTINEL];
7          if (previous == address(0) || previous == SENTINEL) return
               inviteesOrInviters;
8
9          assembly {
10             // Calculate the storage slot for invitationLinkedList[
                   inviterOrInvitee]
11             mstore(0, inviterOrInvitee)
12             mstore(0x20, invitationLinkedList.slot)
13             let inviterOrInviteeSlot := keccak256(0, 0x40)
14             // Start with the first node from solidity
15             let inviteeOrInviter := previous
16             // While inviteeOrInviter != SENTINEL
17             for {} iszero(eq(inviteeOrInviter, 0x01)) {} {  // @audit-ok
18                 // Increase free memory pointer by 0x20 for the new element
19                 mstore(0x40, add(mload(0x40), 0x20))
20                 // Increment array length
21                 mstore(inviteesOrInviters, add(mload(inviteesOrInviters), 0x01)
                       )
22                 // Store the new element in array
23                 mstore(add(inviteesOrInviters, mul(mload(inviteesOrInviters), 0
                       x20)), inviteeOrInviter)  // @audit this is not memory-safe
24                 // @audit-info the array is stored at 0x60
25                 // Compute the storage slot of invitationLinkedList[
                       inviterOrInvitee][inviteeOrInviter]
26                 mstore(0, inviteeOrInviter)
27                 mstore(0x20, inviterOrInviteeSlot)  // @audit useless mstore in
                        the loop
```

```
                let nextSlot := keccak256(0, 0x40)

                // Move to next node
                inviteeOrInviter := sload(nextSlot)
            }
        }
    }
```

Increasing the size of the array this way will overwrite anything in the memory stored after the array, which is why solidity does not have a `.push()` function for dynamic size memory arrays.

Dynamic arrays point to the zero slot `0x60` when not initialized. That means the array `inviteesOrInviters` declared in `returns (address[] memory inviteesOrInviters)` is stored continuously from `0x60` onwards. If anything is in memory after `0x60`, it will be overwritten with the content of `inviteesOrInviters`.

Since the function is declared as `internal`, it uses the same memory context than its caller. Calling this function from anywhere that has elements stored in memory will overwrite them with `inviteesOrInviters`.

Luckily, it is only called in a context where memory is unused. Luckily as well, only two words of memory are used before the array is being manipulated so scratch space is used for them and they are not used after the assembly block.

In any case, the reserved memory address `0x60` is written to and not reset to zero at the end of the assembly block which makes the assembly block not safe. The memory `0x60` is reserved by solidity for uninitialized arrays and should remain zeroed.

Impact

The function `_getInviteesOrInviters()` must not be called in a context where memory is used or the resulting behaviour may be unexpected. No dynamic arrays should be used after the function `_getInviteesOrInviters` has been called.

The fact that the memory pointer is increased gives a false sense of security to the callers of this functions that may encounter sneaky bugs.

Code Snippets

https://github.com/aboutcircles/circles-invitation-escrow/blob/9f1e79203ba1c90a99a5237ce3d29165dff90729/src/InvitationEscrow.sol#L411-L444

Recommendation

Store the array at the memory location pointed by the free memory pointer instead of leaving it at `0x60`. Otherwise, avoid using assembly.

Alternatively, comment that this function is unsafe and breaks memory layout.

Response

The code was updated in commit `0x4dc2fa3` to store the array at the memory location pointed by the free memory pointer before filling it.